

神经网络与深度学习

[本页PDF](#)

绪论

神经网络的发展历程：

- 第一代：感知机
- 第二代：多层感知器的 BP 反向传播算法
- 第三代：深度学习

神经网络结构

- 前馈网络
- 记忆网络
- 图网络

人工神经网络

- 神经元激活规则
- 网络的拓扑结构
- 学习算法

机器学习

什么是机器学习：通过算法使得机器能从大量数据中学习规律从而对新的样本做决策

机器学习三要素：模型、学习准则、优化

深度学习三步骤：定义网络、损失函数、优化

常见三类机器学习问题：分类、回归、聚类

常见的机器学习类型：监督学习、无监督学习、强化学习

随机梯度下降法：每次迭代只选择一个样本来更新网络参数

- 输入训练集、验证集、学习率 α
- 随机初始化
- 循环直到模型在验证集的错误率不再下降（或小于某一个阈值）
 - 对训练集样本进行随机排序
 - 遍历每一个样本
 - 根据选取的样本更新参数
- 输出模型参数 θ
- 伪代码

输入：训练集 $\mathcal{D} = \{x^{(n)}, y^{(n)}\}_{n=1}^N$ ，验证集 $\mathcal{V}$ ，学习率 α

1. 随机初始化 θ

2. repeat

- 对训练集 $\mathcal{D}$ 的样本进行随机排序
- for $n = 1 \dots N$ do
 - 从训练集 $\mathcal{D}$ 选取样本 $(x^{(n)}, y^{(n)})$
 - 更新参数: $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; x^{(n)}, y^{(n)})}{\partial \theta}$
- end

3. until 模型 $f(x; \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降

4. 输出 θ

算法 2.1 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

1 随机初始化 θ ;

2 **repeat**

3 对训练集 $\mathcal{D}$ 中的样本随机排序;

4 **for** $n = 1 \dots N$ **do**

5 从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

6 $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$; // 更新参数

7 **end**

8 **until** 模型 $f(\mathbf{x}; \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降;

输出: θ

机器学习模型的几个评价指标

预测结果为 $\hat{y}$, 真实结果为 y

- 准确率: 预测正确的样本占总样本比例

$$\mathcal{A} = \frac{1}{N} \sum_{n=1}^N I(y^{(n)} = \hat{y}^{(n)})$$

- 错误率: 预测错误的样本占总样本比例

$$\begin{aligned} \mathcal{E} &= 1 - \mathcal{A} \\ &= \frac{1}{N} \sum_{n=1}^N I(y^{(n)} \neq \hat{y}^{(n)}) \end{aligned}$$

- 精确率: 预测为正例占真正是正例的比例

$$P = \frac{TP}{TP + FP}$$



- 召回率：真是正例占被模型预测为正例的比例

$$R = \frac{TP}{TP + FN}$$

精确率和召回率详见[统计学习方法](#)

目标函数

- 经验风险最小化

$$R(w) = \frac{1}{2} \|y - x^T w\|^2$$

- 结构风险最小化

$$R(w) = \frac{1}{2} \|y - x^T w\|^2 + \frac{1}{2} \lambda \|w\|^2$$

- 最大似然估计：等价于经验风险最小化

$$p(y|x; w, \sigma) = \prod_{n=1}^N N(y^{(n)}; w^T x^{(n)}, \sigma^2)$$

- 最大后验估计：等价于结构风险最小化
- 验证岭回归的解：对结构风险求导即可

$$w^* = (XX^T + \lambda I)^{-1} Xy$$

- 主成分分析

$$\max_{w_k} w_k^T \Sigma w_k \text{ s.t. } w_k^T w_k = 1 \quad w_k w_j = 0 \quad (j = 1, \dots, k-1)$$

- 线性判别分析

$$\max_w J(w) = \max_w \frac{w^T S_b w}{w^T S_w w}$$

- 支持向量机

$$\max \frac{1}{\|w\|^2} \text{ s.t. } y^{(n)}(w^T x^{(n)} + b) \geq 1$$

前馈神经网络

反向传播算法执行过程

- 输入训练集，验证集，学习率，正则化系数等
- 随机初始化网络参数
- 循环直到模型在验证集的错误率不再下降（或小于某一个阈值）
 - 对训练集样本进行重排序
 - 遍历每一个样本
 - 前馈计算每一层的净输入和激活值

- 反向传播计算每一层梯度
- 更新参数
- 输出模型参数

算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $M_l, 1 \leq l \leq L$.

```

1  随机初始化  $\mathbf{W}, \mathbf{b}$ ;
2  repeat
3      对训练集  $\mathcal{D}$  中的样本随机重排序;
4      for  $n = 1 \cdots N$  do
5          从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;
6          前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;
7          反向传播计算每一层的误差  $\delta^{(l)}$ ;                                // 公式 (4.63)
          // 计算每一层参数的导数
8           $\forall l, \quad \frac{\partial \mathcal{L}(y^{(n)}, \hat{y}^{(n)})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$ ;                // 公式 (4.68)
9           $\forall l, \quad \frac{\partial \mathcal{L}(y^{(n)}, \hat{y}^{(n)})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$ ;                                // 公式 (4.69)
          // 更新参数
10          $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^\top + \lambda \mathbf{W}^{(l)})$ ;
11          $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$ ;
12     end
13 until 神经网络模型在验证集  $\mathcal{V}$  上的错误率不再下降;
    输出:  $\mathbf{W}, \mathbf{b}$ 

```

反向传播算法的推导

设第 l 层

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

损失函数是

$$\mathcal{L}(y, \hat{y})$$

要求的是

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$$

1. 对某个权重 w_{ij}^l , 使用链式法则

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial w_{ij}^{(l)}}$$

由于

$$z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

因此

$$\frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} = a_j^{(l-1)}$$

所以

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}} a_j^{(l-1)}$$

这时候定义

$$\delta_i^{(l)} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}}$$

所以

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

矩阵形式为

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

2. 推导偏置梯度

因为

$$z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

因此

$$\frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$$

于是有

$$\frac{\partial \mathcal{L}}{\partial b_i^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}} = \delta_i^{(l)}$$

写成矩阵形式为

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

3. 推导 $\delta^{(l)}$

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$$

通过链式法则

$$\frac{\partial \mathcal{L}}{\partial z^{(l)}} = \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \frac{\partial \mathcal{L}}{\partial z^{(l+1)}}$$

逐个看

$$\frac{\partial \mathcal{L}}{\partial z^{(l+1)}} = f'_l(z^{(l+1)})$$

$$\frac{\partial \mathcal{L}}{\partial z^{(l+1)}}$$

且

$$z^{(l+1)} = W^{(l+1)} a^{(l)} + b^{(l+1)}$$

所以

$$\frac{\partial z^{(l+1)}}{\partial a^{(l)}} = (W^{(l+1)})^T$$

而

$$\frac{\partial \mathcal{L}}{\partial z^{(l+1)}} = \delta^{(l+1)}$$

最后

$$\delta^{(l)} = f'_l(z^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)})$$

CNN

特征

- 局部连接
- 权值共享
- 空间或时间上的次采样

RNN

反向传播算法推导

每一步

$$z_t = U h_{t-1} + W x_t + b h_t = f(z_t)$$

输出为

$$o_t = V h_t + c$$

损失为

$$L = \sum_{t=1}^T L_t$$

要求的梯度为

$$\frac{\partial L}{\partial U} \quad \frac{\partial L}{\partial W} \quad \frac{\partial L}{\partial b}$$

定义误差项

$$\delta_{t,k} = \frac{\partial L_t}{\partial z_k}$$

长程依赖问题：**循环神经网络在时间维度上非常深**，包括梯度消失和梯度爆炸，可以使用**循环边改为线性依赖关系和增加非线性**来改进。

- 梯度消失：网络层数很深且导数小于1，经过每一层网络后导数不断衰减，造成梯度衰减
 - 改进模型，循环边改为线性依赖关系和增加非线性
- 梯度爆炸：网络层数很深且导数大于1，经过每一层网络后导数不断被放大，造成梯度爆炸
 - 使用权重衰减或梯度裁剪

应用到机器学习的模式

- 序列到类别
- 同步的序列到序列
- 异步的序列到序列

网络优化与正则化

梯度下降有哪三种

- 批量梯度下降
- 小批量梯度下降
- 随机梯度下降

神经网络优化的改善方法有**网络优化和网络正则化**两大类

小批量梯度下降：每次选取K个样本对模型参数进行更新，影响其的主要因素有

- 批量大小K
- 学习率
- 梯度估计

参数初始化方法

- 预训练初始化
- 随机初始化
 - **神经网络初始化为什么不能将参数全设置为0**：将参数都初始化为0导致第一遍前向计算时中间值都相同，导致反向传播权重更新也相同，导致神经元没有区分性，称为**对称权重现象**
- 固定值初始化

神经网络优化算法

- 学习率调整
 - 固定衰减学习率
 - 周期性学习率
 - 自适应学习率
- 梯度估计修正：动量法
- 综合方法：Adam = 动量法 + RMSProp

神经网络正则化方法

- L1、L2正则化
- 权重衰减
- 早停
- Dropout
- 数据增强
- 标签平滑

超参数优化方法

- 网格搜索
- 随机搜索
- 贝叶斯优化
- 动态资源分配
- 神经架构搜索

不能在循环神经网络中的循环连接上直接应用丢弃法：这样会损害循环网络在时间维度上记忆能力，导致信息流断裂，模型无法理解长序列依赖关系

L1、L2正则化目标函数

$$\theta^* = \argmin_{\theta} \frac{1}{N} \sum_{n=1}^N L(y^{(n)}, f(x^{(n)}; \theta)) + \lambda l_p(\theta)$$

神经网络优化的改善方法

- 使用更有效的优化算法来提高梯度下降优化方法的效率和稳定性，比如动态学习率调整、梯度估计修正
- 使用更好的参数初始化方法、数据预处理方法来提高优化效率
- 修改网络结构来得到更好的优化地形，比如使用ReLU激活函数、残差连接、逐层归一化等
- 使用更好的超参数优化方法

RMSProp

维护梯度平方的累积变量 G_t ，梯度为 g_t

$$G_t = \beta G_{t-1} + (1 - \beta) g_t^2$$
$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{G_t} + \epsilon} g_t$$

Momentum

维护一个速度变量 v_t

$$\begin{aligned}v_t &= \beta v_{t-1} + (1 - \beta)g_t \\ \theta_t &= \theta_{t-1} - \alpha v_t\end{aligned}$$

Adam算法

- 结合了RMSProp和动量法
 - 动量法：使用了梯度的一阶矩
 - RMSprop：使用了梯度的二阶矩

计算移动平均（一阶矩和二阶矩）

$$\begin{aligned}M_t &= \beta_1 M_{t-1} + (1 - \beta_1)g_t \\ G_t &= \beta_2 G_{t-1} + (1 - \beta_2)g_t \odot g_t\end{aligned}$$

偏差修正

$$\begin{aligned}\hat{M}_t &= \frac{M_t}{1 - \beta_1^t} \\ \hat{G}_t &= \frac{G_t}{1 - \beta_2^t}\end{aligned}$$

更新参数

$$\Delta \theta_t = - \frac{\alpha}{\sqrt{\hat{G}_t} + \epsilon} \hat{M}_t$$

逐层归一化

- 批量归一化（Batch Normalization）：不同样本的同一维度上进行归一化
- 层归一化（Layer Normalization）：同一样本的不同维度进行归一化
- 权重归一化
- 局部响应归一化
- 目的：更好的尺度不变性、更平滑的优化地形

无监督学习

典型的无监督学习问题分类

- 无监督特征学习
- 密度估计
- 聚类

主成分分析

- 优化函数：

第一主成分优化问题

$$\max_{w_1} w_1^T \Sigma w_1 \text{ s.t. } w_1^T w_1 = 1$$

第k主成分优化问题

\$\$

$$\begin{aligned} \max_{w_k} \quad & w_k^T \Sigma w_k \\ \text{s.t.} \quad & w_k^T w_k = 1, \quad w_k^T w_j = 0 \quad (j = 1, \dots, k-1) \end{aligned}$$

\$\$

- 求解过程：使用拉格朗日乘子法，对 w_1 求导，令导数为0

线性编码

- 表达式
 - 输入样本 x 可以由基向量 A 和字典 Z 表示
 - 其中，基向量 $A = [a_1, \dots, a_m]$ ，编码向量是 $Z = [z_1, \dots, z_m]^T$

解码

$$x = \sum_{m=1}^M z_m a_m = AZ$$

编码

$$z = \sum_{m=1}^M x_m a_m = A^T x$$

稀疏编码

- 优点
 - 降低计算量
 - 可解释性好
 - 能实现特征的自动选择
- 目标函数
 - 其中 ρ 是稀疏性衡量函数， η 是一个超参数，用来控制稀疏性的强度

$$\mathcal{L}(A, Z) = \sum_{n=1}^N (\|x^{(n)} - Az^{(n)}\|^2 + \eta \rho(z^{(n)}))$$

- 训练过程

1. 固定基向量 A ，对于每个输入 $x^{(n)}$ ，计算其对应的最优编码

$$\min_{z^{(n)}} \|x^{(n)} - Az^{(n)}\|^2 + \eta \rho(z^{(n)}), \forall n \in [1, N]$$

2. 固定上一步得到的编码 $\{z^{(n)}\}_{n=1}^N$ ，计算其最优的基向量

$$\min_A \sum_{n=1}^N (\|x^{(n)} - Az^{(n)}\|^2) + \lambda \frac{1}{2} \|A\|^2$$

编码器

自编码器

- 分为编码器和解码器
- 学习目标是重构错误（最小化重构误差）
- 和主成分分析在学习目标上完全一致：最小化重构误差
 - 在特定条件下，两者是同构的
 - 自编码器是主成分分析的**非线性泛化**
- 三种变体
 - 稀疏自编码器
 - 堆叠自编码器
 - 降噪自编码器

稀疏自编码器

- 目标函数
 - 其中 W 是自编码器中的参数
 - 不仅要求还原输入，还需要学习到稀疏且有意义的特征表示，且引入正则化进行参数惩罚

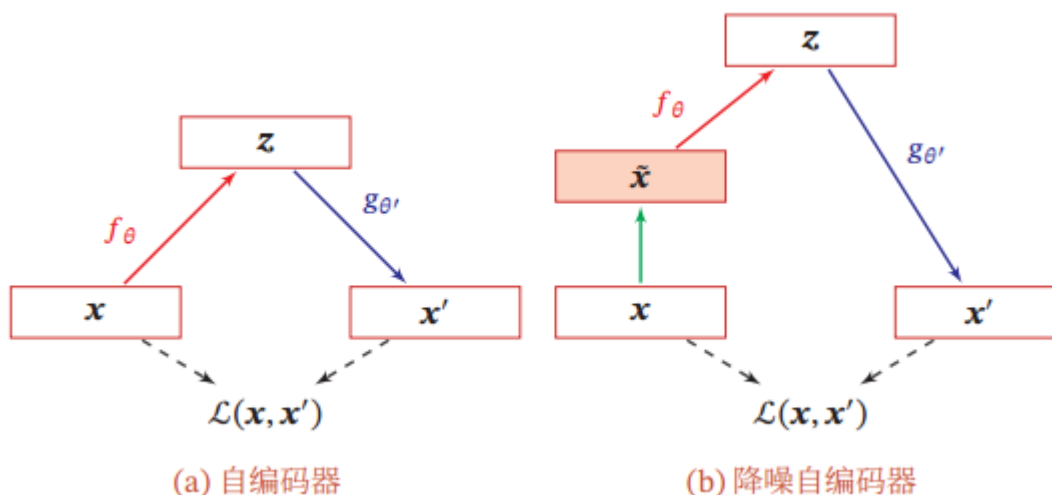
$$\mathcal{L} = \sum_{n=1}^N \|x^{(n)} - x'^{(n)}\|^2 + \eta \rho(z^{(n)}) + \lambda \|W\|^2$$

堆叠自编码器

- 将编码器进行逐层堆叠，训练一个深度的自编码器
- 采用**逐层训练**的方式更新参数

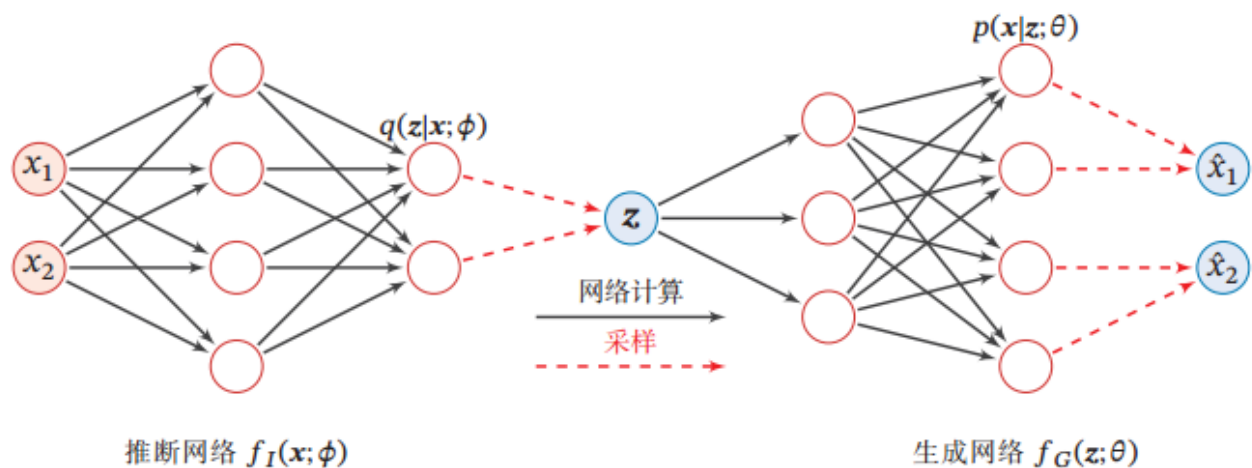
降噪自编码器

- 思想：通过引入噪声来学习更鲁棒性的数据编码，并提高模型的泛化能力
- 结构：在原始输入 x 中引入噪声，交由编码器还原
- 学习目标：要求网络能够对输入样本进行去噪，从含有噪声的样本中恢复原始数据



变分自编码器

- 结构：



- 目标函数：最小化KL散度

$$\phi^* = \arg \min_{\phi} \text{KL} (q(z|x; \phi), p(z|x; \theta))$$

模型独立的学习方法

- 集成学习
- 协同学习
- 多任务学习
- 迁移学习
- 终身学习
- 元学习

注意力机制

组成部分

- 打分函数
- 提取机制
- 聚合机制

四种注意力机制

- 软性注意力：通过计算相似度作为权重对输入加权求和，计算完相似度之后一般用softmax
 - 打分函数： $s(x, q)$ ，对输入 x 和查询 q 计算相似度
 - 信息提取： $s(x, q) \cdot x$ ：用相似度作为连续权重对输入进行加权
- 硬性注意力：只选取相似度最高的那个输入
 - 打分函数： $I[s(x, q); \lambda]$ ，结合重参数技巧或阈值判断
 - 信息提取：只选取相似度最高的那个输入
- 键值对注意力：将输入拆分为 k 用于计算相似度， v 用于存储实际内容
 - 打分函数： $s(k, q)$ ，仅用 k 和 v 计算匹配度
 - 信息提取： $s(k, q) \cdot v$ ，用 k 和 q 的相似度加权对应的 v
- 多头注意力：将Query、Key、Value通过不同线性投影拆分为 h 组
 - 打分函数： $s(k_i, q_i)$
 - 信息提取： $s(k_i, q_i) \cdot v_i$ ，之后进行拼接

	软性注意力机制	硬性注意力机制	键值对注意力机制	多头注意力机制	
打分函数	$s(x, q)$	$I[s(x, q); \lambda]$	$s(k, q)$	$s(k, q_i)$	
信息提取机制	$s(x, q) \cdot x$	$argmax$	$s(k, q) \cdot v$	$s(k, q_i) \cdot v$	
聚合机制	加和	$argmax$	加和	拼接, 加和	

Transformer的注意力机制是基于软性注意力的键值对模式

打分函数

- 加性模型
- 点积模型
- 缩放点积模型
- 双线性模型

加性模型

$$s(\boldsymbol{x}, \boldsymbol{q}) = \boldsymbol{v}^\top \tanh(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{U}\boldsymbol{q}),$$

点积模型

$$s(\boldsymbol{x}, \boldsymbol{q}) = \boldsymbol{x}^\top \boldsymbol{q},$$

缩放点积模型

$$s(\boldsymbol{x}, \boldsymbol{q}) = \frac{\boldsymbol{x}^\top \boldsymbol{q}}{\sqrt{D}},$$

双线性模型

$$s(\boldsymbol{x}, \boldsymbol{q}) = \boldsymbol{x}^\top \boldsymbol{W}\boldsymbol{q},$$

任务相关性越大，分值越大

自注意力计算过程

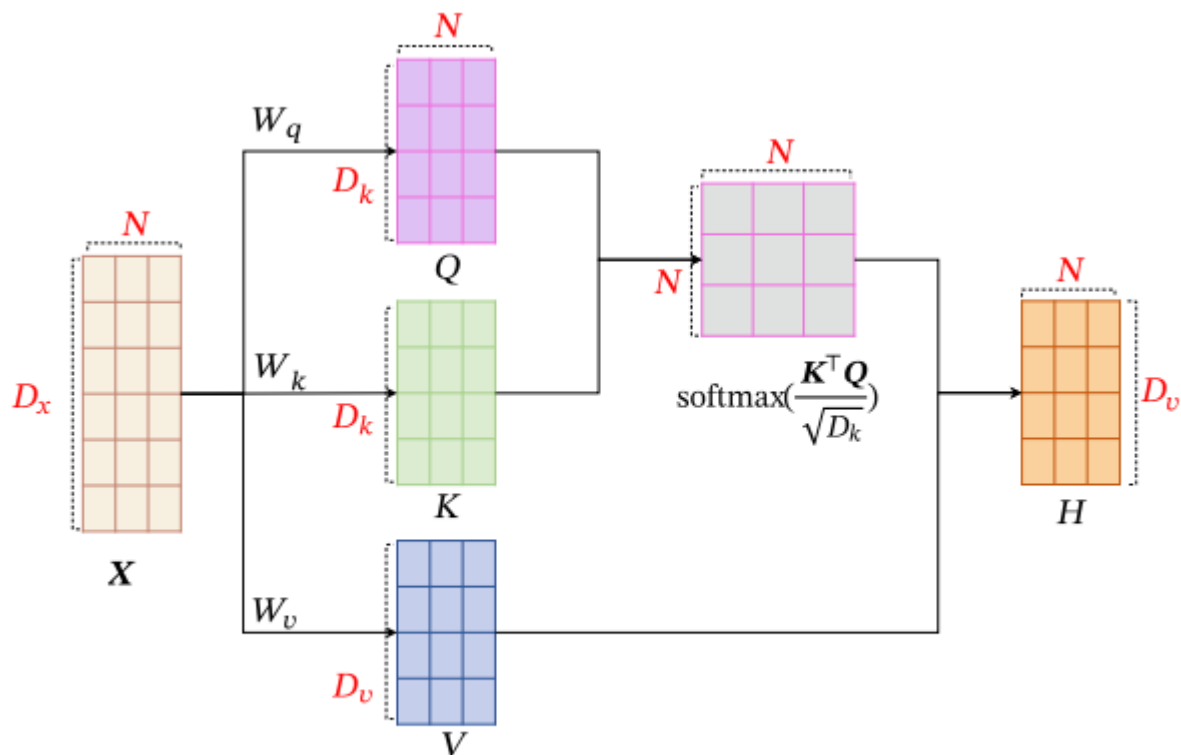


图 8.4 自注意力模型的计算过程

自注意力机制

自注意力机制指的是Q、K、V都来自输入序列本身

当神经网络要处理变长的输入序列时，通常使用循环网络或者卷积网络进行编码得到一个相同长度的输出序列

建立非局部的依赖关系

- 全连接模型
- 自注意力模型

具体请看[Transformer](#)

概率图模型

贝叶斯网络（有向图模型/信念网络）

- 局部马尔可夫性质：每个随机变量在给定父节点的情况下，条件独立于它的非后代节点

必考

$$p(a, b, c) = p(a)p(b|a)p(c|a, b)$$

$$p(x_1, \dots, x_7) = p(x_1)p(x_2)p(x_3)p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_3)p(x_6|x_4)p(x_7|x_4, x_5)$$

基于隐变量的学习和推断

点估计

- MLE：最大似然
- MAP：最大估计

生成模型：包含密度估计和采样两个步骤

含隐变量的参数学习

- 可以看到隐变量：直接优化 $P(X, Z)$
- 看不到隐变量：优化 $P(X) = \int P(X, Z)dZ$ ，但这个积分不好算，因此要用EM算法

EM算法：解决已知观测变量 X 但对于隐变量 Z 未知的问题

- E步（期望步）：使用已知模型参数 θ 和观测变量 X 估计隐变量 Z
 - 估计的是隐变量的后验分布 $q(Z|X, \theta)$ ，并让这个分布接近于真实后验分布 $p(Z|X, \theta)$
 - 精确EM：可以直接得到真实后验分布，直接让 $q(Z|X, \theta)$ 等于 $p(Z|X, \theta)$
 - 其它：无法直接算出真实后验分布，让 $q(Z|X, \theta)$ 近似真实后验概率分布
 - 注意：真实后验概率分布无法直接求出，这里的近似是**间接近似**，也就是在M步最大化似然的同时间接让 $q(Z)$ 和 $p(Z|X, \theta)$ 的KL散度减小
- M步（最大化步）：使用E步估计得到的隐变量 Z 和观测数据 X 重新拟合模型参数 θ

EM算法推导

目标是最大化 $P(X|\theta)$

可以转换为

$$P(X|\theta) = \int P(X, Z|\theta)dz$$

并且有

$$P(X|\theta) = \frac{P(X, Z|\theta)}{P(Z|X, \theta)}$$

因此

$$\begin{aligned}\log P(X|\theta) &= \log \frac{P(X, Z|\theta)}{P(Z|X, \theta)} \\ &= \log P(X, Z|\theta) - \log P(Z|X, \theta) \\ &= \log \frac{P(X, Z|\theta)}{q(z)} - \log \frac{P(Z|X, \theta)}{q(z)}\end{aligned}$$

两边同时乘以 $q(z)$ 并对 z 积分

$$\begin{aligned}P(X|\theta) &= \int q(z) \log P(X|\theta) dz \\ &= \int q(z) \log \frac{P(X, Z|\theta)}{q(z)} - \int q(z) \log \frac{P(Z|X, \theta)}{q(z)} \\ &= \underbrace{E_{z \sim q(z)} \left[\log \frac{P(X, Z|\theta)}{q(z)} \right]}_{\text{LowerBound}} + \underbrace{KL(q(z) || P(Z|X, \theta))}_B\end{aligned}$$

\$\$

因为 $q(z)$ 是在E步固定的，因此 $E_{z \sim q(z)}[-\log q(z)]$ 与 θ 无关

因此对下界B最大化本质是在对

$$E_{z \sim q(z)}[\log P(X, Z|\theta)]$$

最大化

可以对下界B进行进一步变形

由于

$$P(X, Z|\theta) = P(X|Z, \theta)P(Z|\theta)$$

但在VAE里面，默认 $P(Z)$ 服从正态分布，不受 θ 的影响

因此

$$\begin{aligned} E_{z \sim q(z)}[\log \frac{P(X, Z|\theta)}{q(z)}] &= E_{z \sim q(z)}[\log \frac{P(X|Z, \theta)P(Z|\theta)}{q(z)}] \\ &= E_{z \sim q(z)}[\log P(X|Z, \theta)] - KL(q(z)||P(Z|\theta)) \end{aligned}$$

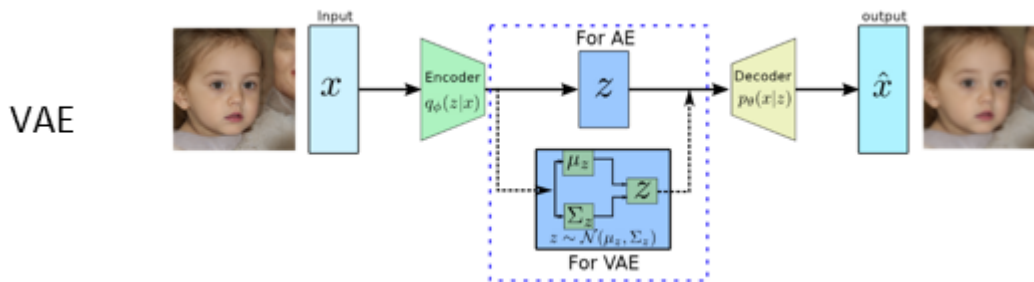
MCMC和变分推断：为了解决E步中真实后验分布难以计算的问题

- **变分推断：**在后验分布难以计算的情况下，引入一个分布 $q(Z)$ 近似真实后验，在M步里面**间接**使他们俩的KL散度最小
 - 给分布 $q(Z)$ 引入参数 ϕ 形成 $q(Z|\phi)$
 - 原EM优化方法
 - 通过优化 ϕ 以增大下界B，**间接**使得近似后验接近真实后验
 - 新的EM优化方法
 - E步优化 ϕ ，使得下界B最大，**间接**使得近似后验接近真实后验
 - M步优化 θ ，使得下界B最大
- **MCMC：**从后验分布中采样，用样本形成的经验分布来近似后验，这些样本随后可以用样本平均来近似M步里的积分或期望

VAE

本质上是使用神经网络实现变分推断

- 目标：总损失 = 重建损失 + KL 散度损失（对应上面EM算法推导的下界B）
 - 重建损失要求解码的图像和原始输入尽量接近
 - KL散度损失要求**近似的后验概率要尽可能接近高斯分布**
 - **注意：**变分推断是引入一个**分布近似真实后验**，在VAE里面也不例外，这里的KL散度损失只是为了让这个近似后验不要那么乱，接近高斯分布好采样和计算
- 隐变量： Z 是一个概率分布
- 编码器：为一个神经网络，输出隐变量 Z 的概率分布（通常为高斯分布），输出 μ 和 σ
- 解码器：接受采样的 Z ，生成图像



VAE架构

VAE方程

- 编码器: $q(z|g_\phi(x))$
- 解码器: $p(x|f_\theta(z))$
- 模型希望似然概率大的同时, 让隐变量分布接近真实后验分布

$$\begin{aligned} \log p(x|\theta) - KL[q(z|g_\phi(x))||p(z|x, \theta)] &= E_{q(z|g_\phi(x))}[\log p(x|f_\theta(z))] - KL[q(z|g_\phi(x))||p(z)] \\ &= \mathcal{L}_2(\theta, \phi|x^{(i)}) + \mathcal{L}_1(\phi|x^{(i)}) \end{aligned}$$

即

$$\mathcal{L}(\theta, \phi|x^{(i)}) = \mathcal{L}_2(\theta, \phi|x^{(i)}) + \mathcal{L}_1(\phi|x^{(i)})$$

更新方式为随机梯度下降

VAE流程

$$x \xrightarrow{\phi} z \xrightarrow{\theta} \hat{x}$$

参数更新

- 通过**重参数化**解决对隐变量分布进行采样后梯度无法回传的问题
 - 想从一个正态分布采样, 直接采样梯度无法回传
 - 因此先采样一个随机噪声 ϵ , 通过 $z = \mu_\phi(x) + \sigma_\phi(x)\epsilon$ 保证梯度可以回传

解码器: 梯度由重建项提供, **不需要重参数化**

$$\mathcal{L}_2 = E_{q(z|g_\phi(x))}[\log p(x|f_\theta(z))]$$

- 实际训练时用采样近似

$$\mathcal{L}_2 \approx \frac{1}{L} \sum_{l=1}^L \log p(x|f_\theta(z^{(l)}))$$

- 因此解码器的梯度为

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{1}{L} \sum_{l=1}^L \frac{\partial \log p(x|f_\theta(z^{(l)}))}{\partial \theta}$$

编码器: 梯度由重建项和KL项提供

- 重建项的梯度需要使用重参数化保证梯度回传

$$\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial \mathcal{L}_1}{\partial \phi} + \frac{\partial \mathcal{L}_2}{\partial \phi}$$

即

$$\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial \mathcal{L}_1}{\partial \phi} + \frac{1}{L} \sum_{l=1}^L \frac{\partial \log p(x|f_{\theta}(\tilde{g}_{\phi}(\epsilon^{(l)}, x)))}{\partial \phi}$$

深度生成模型

GAN

网络架构

- 判别网络：区分输入是真实数据，还是生成网络生成的假数据
- 生成网络：生成尽可能欺骗判别网络的逼真数据

目标函数

符号	含义
$p_r(x)$	真实数据分布
$p_{\theta}(x)$	生成网络生成的数据服从的分布
$z \sim p(z)$	从噪声分布中采样
$G(z; \theta)$	生成网络生成的假样本
$D(x; \phi)$	判别网络认为 (x) 是真实样本的概率
ϕ	判别网络参数
θ	生成网络参数

- 判别网络：让判别网络对真实样本的预测概率接近1，对假样本的预测概率接近0

$$\max_{\phi} E_{x \sim p_r(x)} [\log D(x; \phi)] + E_{z \sim p(z)} [\log (1 - D(G(z; \theta); \phi))]$$

- 生成网络:生成网络希望自己生成的样本 $G(z; \theta)$ 被判别网络认为是真的

$$\max_{\theta} (E_{z \sim p(z)} [\log D(G(z; \theta); \phi)])$$

- 总体目标函数：本质上是极小极大博弈

$$\min_{\theta} \max_{\phi} E_{x \sim p_r(x)} [\log D(x; \phi)] + E_{z \sim p(z)} [\log (1 - D(G(z; \theta); \phi))]$$

最优判别器

此时固定生成网络，调整判别网络的参数

为了让判别网络目标函数最大，对于每一个x，需要最大化

$$p_r(x) \log D(x) + p_\theta(x) \log (1 - D(x))$$

对 $D(x)$ 求导

$$\frac{p_r(x)}{D(x)} - \frac{p_\theta(x)}{1 - D(x)} = 0$$

可以解得最优解

$$D^*(x) = \frac{p_r(x)}{p_r(x) + p_\theta(x)}$$

可以将最优判别器代回目标函数

$$\begin{aligned} \mathcal{L}(G|D^*) &= E_{x \sim p_r(x)} [\log D^*(x)] + E_{x \sim p_\theta(x)} [\log (1 - D^*(x))] \\ &= E_{x \sim p_r(x)} \left[\log \frac{p_r(x)}{p_r(x) + p_\theta(x)} \right] + E_{x \sim p_\theta(x)} \left[\log \frac{p_\theta(x)}{p_r(x) + p_\theta(x)} \right] \end{aligned}$$

我们定义

$$p_a(x) = \frac{1}{2} (p_r(x) + p_\theta(x))$$

即

$$p_r(x) + p_\theta(x) = 2p_a(x)$$

因此

$$\log \frac{p_r(x)}{p_r(x) + p_\theta(x)} = \log \frac{p_r(x)}{2p_a(x)} = \log \frac{p_r(x)}{p_a(x)} - \log 2$$

另一项同理

$$\log \frac{p_\theta(x)}{p_r(x) + p_\theta(x)} = \log \frac{p_\theta(x)}{2p_a(x)} = \log \frac{p_\theta(x)}{p_a(x)} - \log 2$$

所以目标函数可以写为

$$\begin{aligned} \mathcal{L}(G|D^*) &= E_{x \sim p_r(x)} \left[\log \frac{p_r(x)}{p_r(x) + p_\theta(x)} \right] + E_{x \sim p_\theta(x)} \left[\log \frac{p_\theta(x)}{p_r(x) + p_\theta(x)} \right] \\ &= E_{x \sim p_r(x)} \left[\log \frac{p_r(x)}{p_a(x)} \right] + E_{x \sim p_\theta(x)} \left[\log \frac{p_\theta(x)}{p_a(x)} \right] - 2 \log 2 \\ &= D_{KL}(p_r || p_a) + D_{KL}(p_\theta || p_a) - 2 \log 2 \end{aligned}$$

而JS散度(≥ 0)定义为

$$D_{JS}(p_r || p_\theta) = \frac{1}{2} D_{KL}(p_r || p_a) + \frac{1}{2} D_{KL}(p_\theta || p_a)$$

最终目标函数可以写为

$$\mathcal{L}(G|D^*) = 2D_{JS}(p_r || p_\theta) - 2 \log 2$$

注意此时求出来的目标函数已经使得判别器是最优的了，现在要让目标函数最小，以求得最佳的生成网络

很显然，当JS散度为0，即 $p_r = p_\theta$ (生成器分布等于真实分布)时，最优收敛点为

$$\mathcal{L}(G|D^*) = -2\log 2$$

同时

$$D^*(x) = \frac{1}{2}$$

生成网络的梯度消失

在使用原始的极大极小目标进行训练时，最后的目标函数为

$$\mathcal{L}(G|D^*) = 2D_{JS}(p_r||p_\theta) - 2\log 2$$

如果真实分布 p_r 和生成分布 p_θ 差得很远，那么判别器很容易判定真假，即

$$D(x) \approx 1, \quad x \sim p_r, D(x) \approx 0, \quad x \sim p_\theta$$

当两个分布完全不重叠时（JS散度有上界）

$$D_{JS}(p_r||p_\theta) = \log 2$$

此时损失为

$$\mathcal{L}(G|D^*) = 2\log 2 - 2\log 2 = 0$$

对于生成网络来说，得到的梯度很弱，生成网络无法正常收敛

模型坍塌

实际训练中可能不使用原始的极小极大目标函数，而是使用生成网络的目标函数

$$\mathcal{L}'(G|D^*) = E_{x \sim p_\theta(x)}[\log D^*(x)]$$

代入 $D^*(x)$

$$\begin{aligned}\mathcal{L}'(G|D^*) &= E_{x \sim p_\theta(x)}\left[\log \frac{p_r(x)}{p_r(x) + p_\theta(x)} \cdot \frac{p_\theta(x)}{p_\theta(x)}\right] \\&= -E_{x \sim p_\theta(x)}\left[\log \frac{p_\theta(x)}{p_r(x)}\right] + E_{x \sim p_\theta(x)}\left[\log \frac{p_\theta(x)}{p_r(x) + p_\theta(x)}\right] \\&= -KL(p_\theta||p_r) + E_{x \sim p_\theta(x)}[\log(1 - D^*(x))] \\&= -KL(p_\theta||p_r) + 2D_{JS}(p_r||p_\theta) - 2\log 2 - E_{x \sim p_r(x)}[\log D^*(x)]\end{aligned}$$

后两项与生成网络无关，因此生成网络最终目标是

$$\arg\max_{\theta} \mathcal{L}'(G|D^*) = \arg\min_{\theta} KL(p_\theta||p_r) - 2D_{JS}(p_r||p_\theta)$$

先来理解KL散度的两种形式

- 前向KL散度：以真实分布为主，想让模型覆盖真实分布
 - 如果模型输出的分布不在真实分布中，模型会受到巨大的惩罚
 - 缺陷：模型输出的图像会变得模糊，试图覆盖真实分布的所有地方，导致输出的图像变成四不像

- 逆向KL散度：以模型分布为主，想让模型尽量接近真实分布
 - 如果模型生成了一个样本，但是这个样本不在真实分布中，模型会受到巨大的惩罚
 - 缺陷：模型只输出特定的样本，只要它生成的样本在真实分布里是合理的，它就不关心那些它没生成的区域

再看看我们推导出来的目标函数，其是一个典型的**逆向KL散度**，而这个目标函数会造成**模型崩塌**

$$D_{KL}(p_{\theta}||p_r) = \int p_{\theta}(x) \log \frac{p_{\theta}(x)}{p_r(x)} dx$$

- 情况一：生成器生成了真实分布不存在的东西
 - 此时 $p_{\theta}(x) > 0$, $p_r(x) = 0$ ，那么KL散度会无穷大
 - 说明生成器一旦把概率放到真实数据完全没有的区域，会受到巨大惩罚
- 情况二：生成器生成的样本没有完全覆盖真实分布
 - 此时 $p_r(x) > 0$, $p_{\theta}(x) = 0$ ，那么KL散度为0
 - 说明生成器漏掉真实数据的一些模式，反向 KL 不会强烈惩罚它

改进

可使用 Wasserstein 距离衡量不同分布之间的远近

- 即使两个分布没有重叠或者重叠非常少，Wasserstein距离仍然能反映两个分布的远近

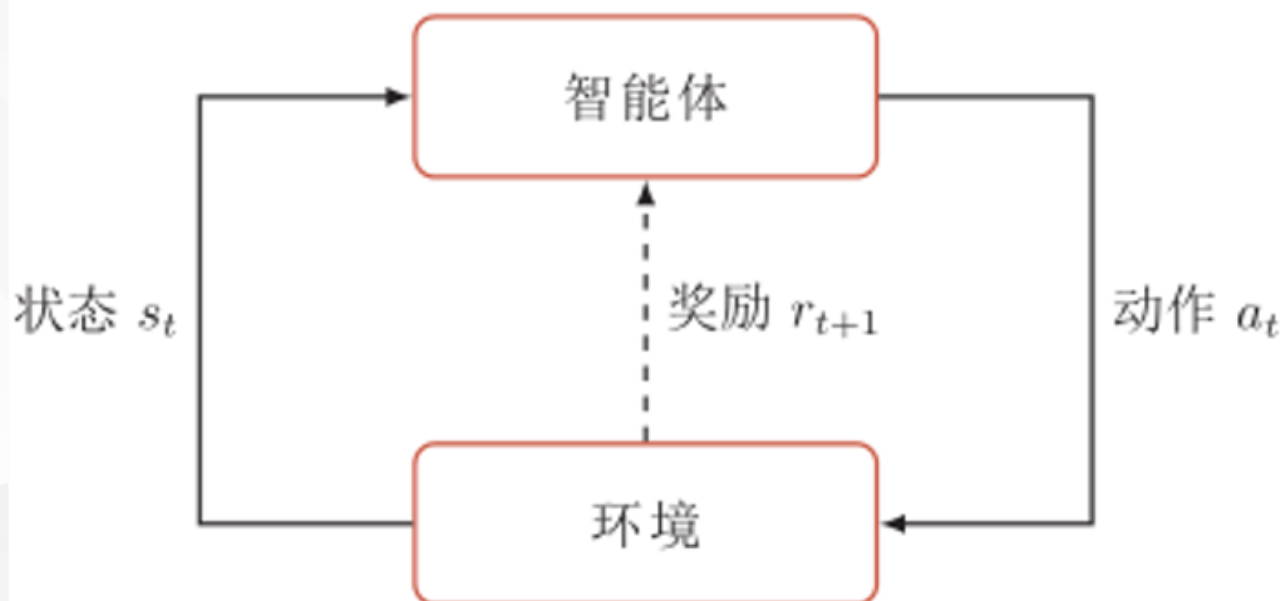
扩散模型

- **前向过程**：对清晰图像每一步加入高斯噪声逐步加噪
- **后向过程**：训练一个网络对噪声图像逐步去噪

强化学习

组成部分

- 智能体
- 环境
- 状态
- 动作
- 奖励



智能体看状态，选动作；环境变状态，给奖励；智能体根据奖励改策略

基本要素

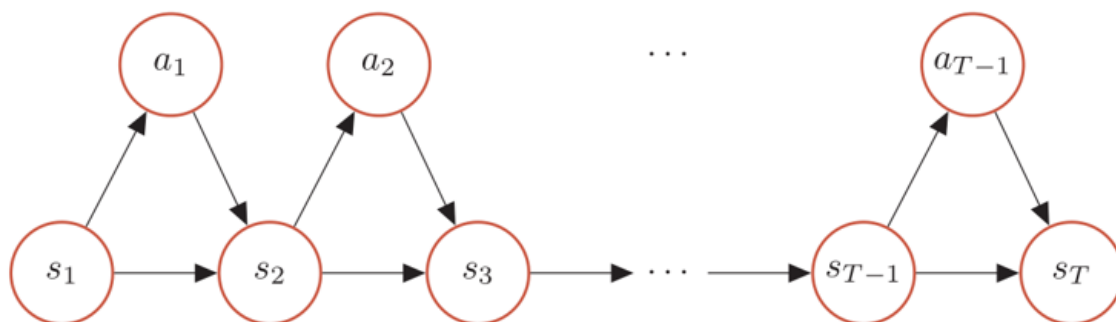
- 环境的状态集合
- 智能体的动作集合
- 策略
- 状态转移概率
- 即时奖励

马尔可夫决策过程的轨迹

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, \dots)$$

• 马尔可夫决策过程的一个轨迹 (trajectory)

$$\tau = s_0, a_0, s_1, r_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T, r_T$$



$$p(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t)p(s_{t+1}|s_t, a_t)$$

总回报

给定策略 $\pi(a|s)$ ，智能体和环境一次交互过程的轨迹 τ 所获得的累计回报成为总回报

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$$

γ 是折扣率

- 当 γ 接近0时，说明模型更在意短期回报
- 当 γ 接近1时，说明模型更在意长期回报

强化学习的目标：学习到一个策略 $\pi_\theta(a|s)$ 来最大化期望回报

$$J(\theta) = E_{\tau \sim p_\theta(\tau)}[G(\tau)] = E_{\tau \sim p_\theta(\tau)}\left[\sum_{t=0}^{T-1} \gamma^t r_{t+1}\right]$$

表示在当前策略产生的轨迹分布下，对轨迹总回报取期望

值函数

状态值函数与状态-动作值函数的区别：

- 状态值函数：当前状态的动作是不确定的，之后按策略 π 走
- 状态-动作值函数：当前状态的动作是确定的，先执行动作之后按策略 π 走

状态值函数

表示：

从状态 s 出发，之后一直按照策略 π 行动，最终能获得的期望总回报（因为可能会有很多条轨迹）

$$V^\pi(s) = E_\pi[G_t | S_t = s]$$

其中 G_t 是从时刻 t 开始的总回报

状态值函数的贝尔曼方程

已知

$$G_t = R_{t+1} + \gamma G_{t+1}$$

代入定义

$$V^\pi(s) = E_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

写成求和形式

1. 在状态 s 下可以采取不同的动作，要对所有动作加权平均
2. 选定动作后，环境也不一定转移到唯一状态，因此要对所有状态加权平均

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) [r(s', a, s) + \gamma V^{\pi}(s')]$$

状态-动作值函数

表示：

在状态 s 下先执行动作 a ，之后按照策略 π 行动，能够获得的期望总回报

$$Q^{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

状态-动作值函数的贝尔曼方程

已知

$$G_t = R_{t+1} + \gamma G_{t+1}$$

代入

$$\begin{aligned} Q^{\pi}(s, a) &= E_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \\ &= E_{\pi}[R_{t+1} + \gamma Q^{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \end{aligned}$$

写成求和形式

1. 当前状态和动作已经确定，因此要对下一状态 s' 进行加权求和
2. 把 $V^{\pi}(s')$ 进行展开，到达下一个状态后可以选择不同的动作，因此要对下一动作 a' 进行加权求和

$$\begin{aligned} Q^{\pi}(s, a) &= \sum_{s'} p(s'|s, a) [r(s', a, s) + \gamma V^{\pi}(s')] \\ &= \sum_{s'} p(s'|s, a) [r(s', a, s) + \gamma \sum_{a'} \pi(a'|s') Q^{\pi}(s', a')] \end{aligned}$$

策略梯度

策略梯度是直接优化策略参数 θ

策略写作：

$$\pi_{\theta}(a|s)$$

表示：

在状态 s 下，按照参数为 θ 的策略选择动作 a

目标函数：

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)}[G(\tau)]$$

即

$$J(\theta) = \sum_{\tau} p_{\theta}(\tau) G(\tau)$$

使用的方法是**梯度上升**

强化学习策略梯度是在最大化 $J(\theta)$ ，因此参数更新使用梯度上升

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

对 θ 求导

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} p_{\theta}(\tau) G(\tau) = \sum_{\tau} \nabla_{\theta} p_{\theta}(\tau) G(\tau)$$

由于

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

代入得

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \sum_{\tau} G(\tau) p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) \\ &= E_{\tau \sim p_{\theta}(\tau)} [G(\tau) \nabla_{\theta} \log p_{\theta}(\tau)] \end{aligned}$$

轨迹概率为

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

环境转移概率 $p(s_{t+1} | s_t, a_t)$ 和初始状态分布不含 θ ，所以求梯度时只剩策略项

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

最终策略梯度行形式为

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [G(\tau) \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

最后可写为

$$\begin{aligned} \nabla_{\theta} J(\theta) &= E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G(\tau_{1:t-1}) + \gamma^t G(\tau_{t:T})) \right] \\ &= E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T (\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \gamma^t G(\tau_{t:T})) \right] \end{aligned}$$

深度Q网络

特点

- 目标网络冻结
- 经验回放

大语言模型

神经网络表示建模 (word2vec)

- 传统分词器的问题：单个词包含的信息量太少，计算量爆炸，稀疏表示难以包含更多的信息

- word2vec：从稀疏的词袋表示，发展到神经网络学习出来的稠密词向量表示
 - 将学习到的**从输入层到隐藏层的权重矩阵**作为词向量

语言模型基本架构（预训练 + 后训练 + 提示工程）

- **输出层**
 - 任务：语言建模、序列到序列（翻译）、分类或非生成任务
 - 把 hidden 层得到的表示转成具体任务需要的结果
- **隐藏层**
 - 模型架构：RNN、LSTM、GRU、Bi-LSTM、Dilated-CNN、Transformer
 - 负责对 embedding 层得到的词向量继续加工，学习词和词之间的关系
- **词嵌入层**
 - 把输入的词或token转换成计算机能处理的向量
 - 采用模型：Word2Vec

表示模型与生成模型

- 表示模型：仅使用Transformer编码器，代表模型BERT
- 生成模型：仅使用Transformer解码器，代表模型GPT

Transformer的预训练

- 预训练基本思路：将预训练方法应用到Transformer上，并在**下游任务中保持Embedding层和Hidden层结构不变**，只针对具体任务进行**输出层的微调**
- 预训练可以分为：Embedding层、Hidden层、Output层
- 两种预训练基本形式（无监督学习）
 - **GPT（自回归语言模型）**：对于每一句话，预测每个位置的下一个Token（LM任务）
 - 使用**掩码注意力**
 - 使用Transformer解码器
 - **BERT**：随机遮住句子中的部分Token，让模型根据上下文预测被挡住的词（MLM任务）
 - 使用**双向注意力**
 - 使用Transformer编码器

语言模型架构流派

- Encoder-only流派：仅包含Transformer编码器，采用双向自注意力机制
 - **代表模型**：Bert、Roberta
 - **适合任务**：文本分类、理解、预测
- Decoder-only流派：仅包含Transformer解码器，采用掩码自注意力机制
 - **代表模型**：OpenAI的GPT系列，Meta的OPT系列
 - **适合任务**：文本生成
- Encoder-Decoder流派：采用序列到序列形式，包含解码器和编码器模块
 - **代表模型**：Google的T5
 - **适合任务**：翻译、对话

总结

:::tabs

@tab:active 绪论 & 机器学习

神经网络结构 神经网络发展历程

机器学习三要素，深度学习的步骤，常见三类机器学习问题，常见的机器学习类型

梯度下降有哪三种

随机梯度下降法步骤

前馈神经网络反向传播算法的推导

使用反向传播算法的随机梯度下降训练过程

机器学习几个评价指标

什么是长程依赖问题、梯度消失、梯度爆炸，分别如何改进

参数初始化方法有哪些，为什么神经网络不能初始化参数为0，会产生什么现象

神经网络正则化方法有哪些

神经网络的优化算法有哪些

神经网络优化的改善方法有哪些

Adma算法的内容

L1、L2正则化优化函数

超参数优化方法有哪些

几种常用的逐层归一化方法 逐层归一化的目标

典型的无监督学习问题可以分为哪几类

模型独立的学习方法有哪些

CNN的特征

RNN应用到机器学习的模式

RNN梯度消失和梯度爆炸如何解决

为什么不能在循环神经网络中的循环连接上直接应用丢弃法

自编码器三种变体

稀疏编码的目标函数和训练过程

自编码器的学习目标 和主成分分析的关系

稀疏自编码器目标函数和含义

堆叠自编码器的结构和训练方式

降噪自编码器的思想、结构、学习目标

稀疏编码的优点

@tab 注意力机制

注意力机制的组成部分

有哪几种注意力机制

打分函数有哪几种

为什么自注意力机制有“自”

多头注意力计算过程

@tab 概率图模型

贝叶斯有向无环图的联合概率密度

MLE和MAP

点估计公式推导

变分推断为什么叫变分

EM算法两个步骤做了什么 原始的和改进的EM算法有什么区别

@tab 生成模型

VAE的架构

VAE的目标函数

VAE编码器、解码器梯度如何计算

GAN的判别网络、生成网络的目标函数 总的目标函数

最优收敛点如何推导、最后损失多少、对应的最佳判别器如何推导、最后值是多少

GAN生成网络梯度消失的问题、模型坍塌的问题是咋来的

扩散模型前向和后向在做什么事情

@tab 强化学习

强化学习流程图 轨迹图

轨迹的联合概率怎么写 智能体和环境一次交互得到的总回报怎么写

强化学习的目标是什么

折扣率表达的含义

状态值函数、状态动作值函数以及其贝尔曼方程

策略梯度公式 最终策略梯度行形式

深度Q网络特点

@tab 大模型

语言模型架构图 架构图每一层的任务及其代表模型

表示模型和生成模型

Transformer预训练思路

gpt Bert预训练基本形式（任务、注意力方式、使用了什么模块）

语言模型架构流派、代表模型、适用的任务